

Mafsal

Ephemeral Zero-Trust Data Shield — encrypted, traceless tunnel for untrusted terminals

GPL v3

Python 3.9+

Experimental

■ **Experimental Software.** Mafsal is in an early experimental phase. Active development is paused for the foreseeable future, but the project is fully open to the community. Fork requests and derivative works are warmly welcomed.

Why "Mafsal"?

Mafsal (𐤌𐤎𐤔𐤕) is the Turkish and Arabic word for **joint** — the anatomical point where two bones meet, articulate, and move in concert while remaining structurally distinct.

The name is deliberate. When you sit at an untrusted kiosk, there is an inherent tension between two things that must coexist: your private data and a machine you do not control. Mafsal is the joint between them — the articulation point that lets them interact without merging. It keeps your session fluid and functional while maintaining a hard structural boundary between what is yours and what belongs to the machine.

A joint does not fuse. It connects, permits movement, and then releases. When your session ends, the joint dissolves — nothing of yours remains in the socket. That is precisely what Mafsal does.

Mafsal in both languages also carries the sense of **detail** or **elaboration** — the part of a structure where complexity lives. Security, done properly, is never a single wall. It is a composition of many small, well-fitted parts: key exchange here, per-packet derivation there, a replay guard at the seam, traffic noise at the edge. Mafsal is a name for that kind of careful articulation.

What is Mafsal?

Mafsal is an open-source **Privacy-by-Design** tunnel system designed to protect your personal and financial data when you must use a computer you do not own or control.

It defends against technically capable adversaries on the local network: packet sniffers, keyloggers, and RAM scrapers. When your session ends, Mafsal leaves no trace on the machine you used.

Project Status

Mafsal is **experimental software**. It has been tested in controlled scenarios and the cryptographic design is sound, but it has not undergone a formal third-party security audit. Use it with that understanding.

Active development is **currently paused** with no roadmap or release schedule. That said, the project is intentionally kept open so that others can pick it up, extend it, and take it wherever they see fit.

If you want to improve Mafsal, the best path is to fork it. Seriously — fork requests make the authors unreasonably happy. Whether you want to add Windows-native support, swap out the tunnel backend, write a

proper test suite, or carry out an independent security review, the door is wide open.

Architecture

```
[Firefox ESR 115+]
| SOCKS5 -> 127.0.0.1:1080 (Kill-Switch active)
v
[mafsal_client.py]   <- Kiosk side (runs in RAM / tmpfs)
| X25519 Ephemeral Key Exchange
| AES-256-GCM + per-packet HKDF sub-key
| ReplayGuard (64-bit sliding window)
| Traffic Shaping (Padding + Jitter + Decoy)
v
[Cloudflare Tunnel]  <- trycloudflare.com (free, no account needed)
v
[mafsal_server.py / mafsal_colab.ipynb] <- Google Colab (exit node)
v
[Target Internet Host]
```

The Cloudflare Tunnel gives the Colab server a public **wss://** address without requiring a domain, static IP, or any Cloudflare account. The URL is ephemeral — it changes every Colab session, which is a feature, not a bug.

How It Works

1. Server side

mafsal_colab.ipynb (or mafsal_server.py) runs on Google Colab and is exposed via a free Cloudflare Tunnel. No account, no configuration, no domain needed.

2. Handshake

mafsal_client.py performs an X25519 Diffie-Hellman exchange authenticated with HMAC-SHA256 over a shared pre-key. A unique ephemeral session key is derived; the pre-key never travels over the wire.

3. Encryption

Every packet is encrypted with AES-256-GCM using a per-packet sub-key derived via HKDF from the session key, a random 8-byte salt, and a monotonic counter. Replaying or reordering packets is detected and rejected by a 64-bit sliding-window ReplayGuard (RFC 4303 style).

4. Traffic shaping

Packets are padded to random lengths, sent with log-normal jitter delays, and supplemented with decoy packets during idle periods. An observer cannot distinguish real traffic from noise.

5. Browser

Firefox is launched with ZeroTrustProfile: WebRTC completely disabled, DNS-over-HTTPS active, SOCKS5 kill-switch on. If the tunnel drops, Firefox goes dark — it cannot reach the internet directly.

6. Session end

The session key is scrubbed from memory, the tmpfs profile disappears, and temporary files are overwritten three times before deletion. No trace remains on disk.

Security Features

Cryptography

Feature	Detail
Key Exchange	X25519 Ephemeral (Forward Secrecy)
Encryption	AES-256-GCM
Key Derivation	HKDF-SHA256 (per-packet sub-key)
Authentication	HMAC-SHA256 (handshake)
Replay Protection	64-bit sliding window (RFC 4303 style)

Traffic Shaping (traffic_shaping.py)

Technique	Purpose
Randomized Padding	Hides fixed packet-size signatures
Human-like Jitter	Log-normal delay mimics real browser behaviour
Decoy Traffic	Random encrypted packets during idle — observers cannot distinguish idle from active

Browser Security (ZeroTrustProfile/)

Setting	Value
WebRTC	Completely disabled (no IP leak)
Kill-Switch	failover_direct = false (internet cut if tunnel fails)
DNS-over-HTTPS	Active (DNS also tunnelled via SOCKS5)
Profile location	RAM / tmpfs (no disk trace)

Requirements

Component	Version
Python	3.9+
Firefox ESR	115+
Google Colab	Free account is sufficient
OS	Linux, macOS, Windows (WSL recommended)

```
pip install -r requirements.txt
```

File Structure

```
mafsal/
■■■ README.md           <- This file
■■■ QUICK_START.md      <- Step-by-step guide
■■■ TECHNICAL_REFERENCE.md <- Cryptographic reference
■■■ CONTRIBUTING.md
■■■ LICENSE              <- GNU GPL v3.0
■■■ requirements.txt
■■■ mafsal_colab.ipynb   <- Colab notebook (server setup)
■■■ mafsal_client.py     <- Kiosk-side SOCKS5 proxy
■■■ mafsal_server.py     <- Colab-side bridge server
■■■ traffic_shaping.py   <- Padding, jitter, decoy module
■■■ launch_zero_trust.sh <- One-command launcher
■■■ ZeroTrustProfile/
    ■■■ prefs.js
    ■■■ distribution/
        ■■■ policies.json
```

Contributing

Pull requests and issues are welcome even during the development pause — they will be reviewed as time allows. Please read CONTRIBUTING.md before submitting.

Areas where community contributions would be especially valuable:

- Windows native support (without WSL)
- Formal test suite for traffic_shaping.py
- Independent security review of the handshake protocol
- Alternative tunnel backends (beyond Cloudflare)
- A persistent-URL tunnel option so the Colab URL does not change on restart

License

Copyright (C) 2026 Mafsal Contributors. This program is free software: you can redistribute it and/or modify it under the terms of the **GNU General Public License** as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

Full license text: <https://www.gnu.org/licenses/gpl-3.0.html> — also included as the LICENSE file in this repository.